

API Faso Yiri

Référence technique complète

Plateforme éducative — Burkina Faso
Production : apieducative.genius-universe.com
Version générée le 06/05/2026

Amendement #10 — Domaines modulaires par pack d'abonnement

Pour : Ousseini, Germain (équipe mobile Flutter) **Date :** 29 avril 2026 **Statut :**  Backend déployé en production
API Base : <https://apieducative.genius-universe.com>

1. Contexte

L'admin peut désormais configurer **quels domaines éducatifs** sont inclus dans chaque pack d'abonnement (Découverte, Essentiel, Famille, Premium Annuel, ou nouveaux packs créés dynamiquement).

Conséquences fonctionnelles

Acteur	Impact
Admin	Crée/modifie les packs et choisit les domaines via l'interface web
Parent	Voit les domaines inclus dans chaque pack avant de souscrire
Enfant	Ne voit que les contenus dont le domaine est inclus dans son abonnement actif

Règle clé : SNAPSHOT à la souscription

Quand un parent souscrit un pack pour son enfant :

- Les domaines actuellement liés à ce pack sont **copiés** dans `abonnement_domaines`
- Si l'admin modifie le pack après → les abonnements **déjà actifs ne changent pas** (ils gardent leurs domaines initiaux jusqu'à expiration)
- Les **nouvelles souscriptions** prennent les domaines actuels du pack

Cas particulier : pack sans configuration

Si un pack n'a aucun domaine configuré (`pack_domaines` vide), c'est traité comme **"tous les domaines disponibles"** (option transition douce). Cela évite de casser les abonnements actifs au déploiement initial.

2. Modifications côté API (déjà déployées)

2.1 Nouvelles tables

```
pack_domaines (type_abonnement_id, domaine_id, date_ajout)
abonnement_domaines (abonnement_id, domaine_id, date_snapshot)
```

2.2 Endpoints à intégrer côté mobile

a) Lister les types d'abonnements (modifié)

GET /api/abonnements/types — désormais inclut `domaines[]` pour chaque pack.

Réponse :

```
{
  "success": true,
  "data": [
    {
      "id": 4,
      "nom": "Premium Annuel",
      "description": "Meilleure offre ! Accès illimité pendant 1 an",
      "prix": "40000.00",
      "devise": "XOF",
      "duree": "ANNUEL",
      "duree_jours": 365,
      "nombre_appareils_max": 5,
      "telechargement_autorise": 1,
      "contenu_premium": 1,
      "domaines": [
        { "id": 1, "nom": "Langues", "icone": "🌐", "couleur": "#3498db" },
        { "id": 2, "nom": "Mathématiques", "icone": "📐", "couleur": "#e74c3c" }
      ]
    }
  ]
}
```

Note : si `domaines: []`, signifie pack non configuré → fallback "tous les domaines".

b) Lister les domaines d'un pack précis

GET /api/abonnements/types/:id/domaines — auth requise (tout utilisateur)

Réponse :

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nom": "Langues",
      "description": "Français, Anglais et langues locales",
      "icone": "🌐",
      "couleur": "#3498db",
      "ordre_affichage": 1
    }
  ]
}
```

c) Lister les domaines accessibles d'un abonnement actif (snapshot)

GET /api/abonnements/:id/domaines — auth requise (propriétaire ou admin)

Réponse : identique à (b)

Utile pour afficher dans l'app parent : "Ton enfant a accès à : [Mathématiques, Langues]"

d) Catalogue contenus (filtrage automatique côté serveur)

GET /api/contenus — aucun changement de payload côté client

Le serveur filtre automatiquement par les domaines de l'abonnement actif de l'enfant connecté.

- Si l'enfant a un abonnement actif **avec snapshot** → catalogue limité aux domaines snapshotés
- Si l'enfant a un abonnement actif **sans snapshot** → catalogue complet (option transition douce)
- Si l'enfant n'a pas d'abonnement actif → catalogue public uniquement

Côté mobile : aucune logique à changer. L'API renvoie déjà la bonne liste.

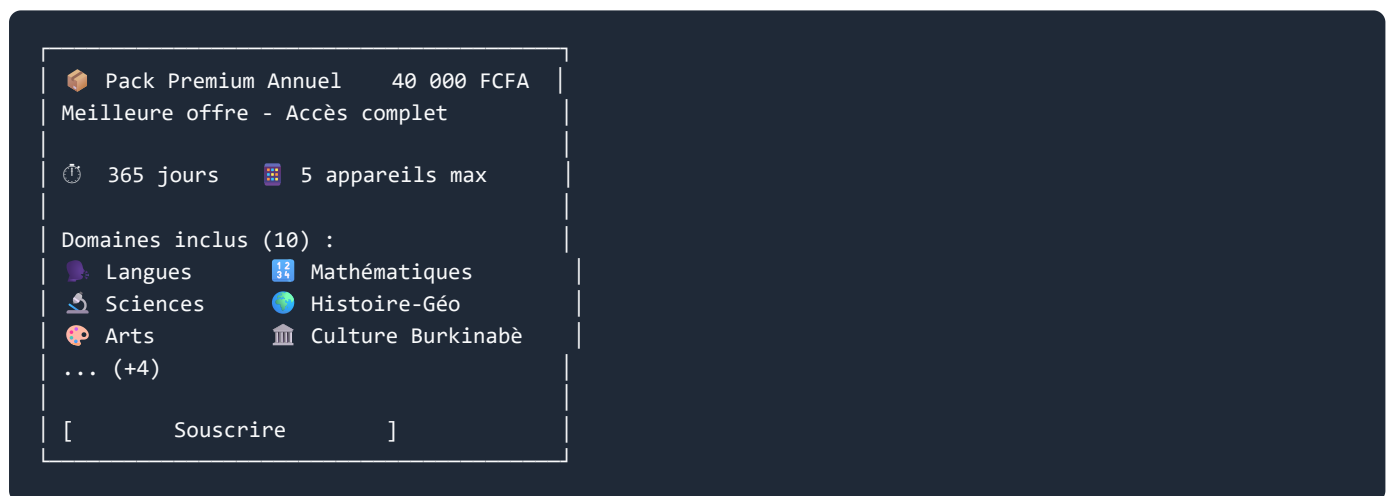
3. Ce qui change côté mobile

3.1 Écran de souscription (parent)

Avant : affichait juste prix + durée + appareils max + checkboxes téléchargement/premium.

Maintenant : afficher aussi la **liste des domaines inclus** pour aider le parent à choisir.

Exemple UI suggéré :



Code Dart suggéré (modèle) :

```

class TypeAbonnement {
    final int id;
    final String nom;
    final String description;
    final double prix;
    final String devise;
    final String duree;
    final int dureeJours;
    final int nombreAppareilsMax;
    final List<DomaineInclus> domaines; // ← NOUVEAU

    TypeAbonnement.fromJson(Map<String, dynamic> json)
        : id = json['id'],
          nom = json['nom'],
          description = json['description'] ?? '',
          prix = double.parse(json['prix'].toString()),
          devise = json['devise'] ?? 'XOF',
          duree = json['duree'],
          dureeJours = json['duree_jours'],
          nombreAppareilsMax = json['nombre_appareils_max'] ?? 1,
          domaines = (json['domaines'] as List? ?? [])
            .map((d) => DomaineInclus.fromJson(d))
            .toList();
}

class DomaineInclus {
    final int id;
    final String nom;
    final String? icone;
    final String? couleur;

    DomaineInclus.fromJson(Map<String, dynamic> json)
        : id = json['id'],
          nom = json['nom'],
          icone = json['icone'],
          couleur = json['couleur'];
}

```

3.2 Écran "Mon abonnement" (parent)

Afficher les domaines réellement accessibles via `GET /api/abonnements/:id/domaines`.

```

final res = await dio.get('/?route=/api/abonnements/$aboId/domaines');
final domaines = (res.data['data'] as List)
  .map((d) => DomaineInclus.fromJson(d))
  .toList();

```

3.3 Catalogue enfant

Aucune modification. L'API filtre déjà côté serveur.

Si l'enfant n'a pas d'abonnement actif, on affiche les contenus publics (comportement inchangé).

3.4 Téléchargement hors-ligne (kiosque)

Le mode kiosque doit pouvoir télécharger **tous** les contenus auxquels l'enfant a accès.

Logique recommandée :

1. Au démarrage, l'app récupère la liste des domaines de l'abonnement actif (`GET /api/abonnements/:id/domaines`)
2. Pour chaque domaine, télécharger tous les contenus publiés
3. Stocker en base locale (Hive/Drift) avec lien `domaine_id`
4. Au lancement offline, montrer ces contenus

Champ obsolète : `est_telechargeable` sur les contenus est conservé en BDD mais doit être ignoré côté mobile (toujours téléchargeable). L'admin n'a plus la possibilité de marquer un contenu non-téléchargeable.

4. Cas limites & tests

4.1 Pack non configuré (fallback)

Comportement : l'API retourne `domaines: []` pour ce pack.

Côté mobile : afficher "Tous les domaines" au lieu de "0 domaines".

```
final label = type.domaines.isEmpty
? 'Tous les domaines disponibles'
: '${type.domaines.length} domaines inclus';
```

4.2 Renouvellement d'abonnement

Au renouvellement (création d'un nouvel abonnement après expiration), le snapshot est **refait** avec les domaines actuels du pack. C'est géré côté serveur, rien à faire côté mobile.

4.3 Modification de pack après souscription

L'admin peut ajouter/retirer des domaines d'un pack à tout moment :

-  N'affecte PAS les abonnements actifs (snapshot conservé)
-  Affecte les futures souscriptions

L'app mobile doit donc rafraîchir périodiquement les domaines de l'abonnement (cache TTL ~24h ou pull-to-refresh).

4.4 Cache côté mobile

Recommandé de **caler** les listes de domaines :

- `GET /api/abonnements/types` → cache 1 heure (rare changement)
 - `GET /api/abonnements/:id/domaines` → cache 24h ou jusqu'à pull-to-refresh
 - Le serveur supporte déjà `_t=timestamp` pour bust le cache Varnish si besoin
-

5. Checklist d'intégration mobile

Phase 1 — Affichage (priorité haute)

- ☐ Mettre à jour le modèle `TypeAbonnement` pour inclure `domaines: List<DomaineInclus>`

- ☐ Afficher la liste des domaines sur l'écran de souscription
- ☐ Gérer le cas `domaines: []` → "Tous les domaines disponibles"

Phase 2 — Visibilité (priorité moyenne)

- ☐ Sur l'écran "Mon abonnement", afficher les domaines accessibles via `GET /api/abonnements/:id/domaines`
- ☐ Sur le profil enfant, afficher un badge avec le nombre de domaines accessibles

Phase 3 — Téléchargement kiosque (priorité haute pour kiosque)

- ☐ Utiliser `GET /api/abonnements/:id/domaines` pour savoir ce qu'il faut télécharger
- ☐ Retirer toute logique liée à `est_telechargeable` côté UI (champ obsolète)
- ☐ Stocker `domaine_id` localement pour pouvoir filtrer offline

Phase 4 — Tests

- ☐ Tester souscription parent → enfant voit uniquement les contenus des domaines inclus
- ☐ Tester pack non configuré → tous les contenus visibles (fallback)
- ☐ Tester modif pack par admin → abonnement actif inchangé
- ☐ Tester renouvellement → nouveaux domaines snapshotés

6. Comptes de test (production)

Mots de passe à demander à Joachim.

Rôle	Email
Admin	admin@plateforme-educative.bf
Gestionnaire	gestionnaire@plateforme-educative.bf
Valideur	valideur@plateforme-educative.bf

Pour tester côté enfant : créer un profil enfant via le compte parent, puis se connecter avec le `codeConnexion` retourné (format `ENF-XXXXXX`).

7. Documentation API complète

La référence API complète (avec tous les endpoints existants) est dans :

- [API_REFERENCE.md](#)
- [API_REFERENCE.pdf](#)

Nouvelle section ajoutée pour cet amendement (section "Domaines par pack").

8. Questions / Support

Pour toute question :

- Contacter Joachim (responsable backend + web)
- Slack/WhatsApp groupe projet
- Issues GitHub : <https://github.com/eaglelax/plateforme-education>

Bon courage pour l'intégration ! 🚀